This is makes Bike class robust of changes to its engine and tire dependency but the consumer of bike class have manage extra lines which he doesn't bother about. The same problems occurs but this time with the consumer of the Bike class.

We can solve the problem of cluttering consumer class with useless imports and initialization by using factory design pattern.

```
import { BikeEngine, BikeTires, Bike } from './bike';

export class BikeFactory {
    createCar() {
        let bike = new Bike(this.createEngine(), this.createTires());
        bike.description = 'Factory Made';
        return bike;}
    createEngine() {return new BikeEngine();}
    createTires() {return new BikeTires();}}
```

Now consumer class will simply import BikeFactory class and delegate the creation of Bike to factory instance.

Factory design patterns really helps with the dependency management but we have to create a separate class just to manage the single class. Imagine project with thousands of classes with minimum 3 to 4 dependencies. We will have that many factory classes to manage which creates more work for us.

The dependency framework is used to solve this issue. The framework provides a way to easily declare dependency of class and have them automatically injected by the framework. Whenever we need the instance of bike we would simply call,

```
let bike = injector.get(Bike);
```

Angular dependency injection

In angular we just have to use Injector decorator to mark the dependency class,

```
import { Injectable } from '@angular/core';

@Injectable()
export class BikeService {
      constructor(BikeEngine engine, BikeTires tires)
}
```